

HIBERNATE

- Hibernate is an Open Source ORM (Object Relational Mapping) Framework used in Java applications to simplify database operations.
- Hibernate acts as a bridge between:
- **Java Application <----> Database**
- Instead of writing SQL queries manually, developers work with Java objects, and Hibernate automatically converts them into SQL queries.
- In JDBC:

```
Connection con = DriverManager.getConnection(...);

PreparedStatement ps =
con.prepareStatement("insert into student values(?,?,?)");
```

```
con.prepareStatement("insert into student values(?,?,?)");
```

ORM : Object Relational Mapping

- **ORM (Object Relational Mapping)** is a technique used to convert data between an object-oriented programming language (such as Java) and a relational database (such as MySQL, Oracle, PostgreSQL).
- In simple terms, ORM acts as a bridge between Java objects and database tables. Instead of writing SQL queries manually, developers can work with Java objects, and the ORM framework automatically handles the database operations.
- In Java applications, data is represented as objects, whereas databases store data in tables consisting of rows and columns.
- This difference is known as the **Object-Relational Impedance Mismatch**.

How ORM Works

1. Create a Java class.
2. ORM maps the class to a database table.
3. Create an object of the class.
4. ORM converts the object data into SQL queries.
5. Data is stored in the database.
6. When data is retrieved, ORM converts table records back into Java objects.

```
<!-- Hibernate -->
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.4.4.Final</version>
</dependency>

<!-- MySQL -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.3.0</version>
</dependency>

<!-- JPA -->
<dependency>
  <groupId>jakarta.persistence</groupId>
  <artifactId>jakarta.persistence-api</artifactId>
  <version>3.1.0</version>
</dependency>
```

```
</dependency>
<version>3.1.0</version>
<artifactId>jakarta.persistence-api</artifactId>
<groupId>jakarta.persistence</groupId>
```

DEPENDENCIES:

<!-- Hibernate -->

<dependency>

<groupId>org.hibernate.orm</groupId>

<artifactId>hibernate-core</artifactId>

<version>6.4.4.Final</version>

</dependency>

<!-- MySQL -->

<dependency>

<groupId>com.mysql</groupId>

<artifactId>mysql-connector-j</artifactId>

<version>8.3.0</version>

</dependency>

<!-- JPA -->

<dependency>

<groupId>jakarta.persistence</groupId>

<artifactId>jakarta.persistence-api</artifactId>

<version>3.1.0</version>

</dependency>

```

<hibernate-configuration>
  <session-factory>

    <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver
    </property>

    <property name="hibernate.connection.url">
      jdbc:mysql://localhost:3306/hibernatedb
    </property>

    <property name="hibernate.connection.username">
      root
    </property>

    <property name="hibernate.connection.password">
      Nithin@01
    </property>

    <property name="hibernate.hbm2ddl.auto">
      update
    </property>

    <property name="hibernate.show_sql">
      true
    </property>

  </session-factory>
</hibernate-configuration>

```

```

<hibernate-configuration>
  <session-factory>

    <property>
      true
    </property>
  </session-factory>
</hibernate-configuration>

```

hibernate.cfg.xml:

```

<hibernate-configuration>
  <session-factory>

    <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver
    </property>

    <property name="hibernate.connection.url">
      jdbc:mysql://localhost:3306/hibernatedb
    </property>

```

```
<property name="hibernate.connection.username">
    root
</property>

<property name="hibernate.connection.password">
    password
</property>

<property name="hibernate.hbm2ddl.auto">
    update
</property>

<property name="hibernate.show_sql">
    true
</property>

</session-factory>
</hibernate-configuration>
```

```

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="student")
public class Student {

    @Id
    private int roll;

    private String name;

    private String email;

    public Student() {
    }

    public Student(int roll, String name, String email) {
        this.roll = roll;
        this.name = name;
        this.email = email;
    }

    public int getRoll() {
        return roll;
    }

    public void setRoll(int roll) {
        this.roll = roll;
    }
}

```

```

}

    this.roll = roll;
    public Student(int roll, String name, String email) {
    }

    return roll;
    public void setRoll(int roll) {

```

Entity:

```

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

```

```

@Entity
@Table(name="student")
public class Student {

```

```

    @Id
    private int roll;

```

```
private String name;
```

```
private String email;
```

```
public Student() {  
}
```

```
public Student(int roll, String name, String email) {  
    this.roll = roll;  
    this.name = name;  
    this.email = email;  
}
```

```
public int getRoll() {  
    return roll;  
}
```

```
public void setRoll(int roll) {  
    this.roll = roll;  
}
```

```
public String getName() {  
    return name;  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
public void setEmail(String email) {  
    this.email = email;  
}
```

```
}
```

```
public class App {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
  
        Configuration config = new Configuration();  
        config.configure();  
        config.addAnnotatedClass(Student.class);  
        SessionFactory sessionFactory = config.buildSessionFactory();  
        Session session = sessionFactory.openSession();  
        Transaction tx = session.beginTransaction();  
  
        Student s1 =  
            new Student(1,  
                        "alex",  
                        "alex@gmail.com");  
        session.persist(s1);  
  
        tx.commit();  
  
        session.close();  
  
        sessionFactory.close();  
  
        System.out.println("Inserted Successfully");  
    }  
}
```

App.java:

```
public class App {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
  
        Configuration config = new Configuration();
```



```
config.configure();
config.addAnnotatedClass(Student.class);
SessionFactory sessionFactory = config.buildSessionFactory();
Session session = sessionFactory.openSession();
Transaction tx =session.beginTransaction();

Student s1 =
    new Student(1,
        "alex",
        "alex@gmail.com");
session.persist(s1);

tx.commit();

session.close();

sessionFactory.close();

System.out.println("Inserted Successfully");

    }
}
```

GET ALL DATA

```
String hql = "from Student";

List<Student> students =
    session.createQuery(hql, Student.class)
        .getResultList();

for(Student s : students) {

    System.out.println("Roll : " + s.getRoll());
    System.out.println("Name : " + s.getName());
    System.out.println("Email: " + s.getEmail());

    System.out.println("-----");
}
}
```

GET BY ID

```
// Fetch student with roll = 1
Student student =
    session.get(Student.class, 1);

if(student != null) {

    System.out.println("Roll : " + student.getRoll());
    System.out.println("Name : " + student.getName());
    System.out.println("Email: " + student.getEmail());

} else {

    System.out.println("Student Not Found");
}
}
```

DELETE BY ID:

```
int id = 1;

Student student =
    session.get(Student.class, id);

if(student != null) {

    session.remove(student);

    System.out.println("Student Deleted Successfully");

} else {

    System.out.println("Student Not Found");

}

tx.commit();
```

```
tx.commit();

}

System.out.println("Student Not Found");
```